# Data Preparation - Snowflake

| | |
|---|---|
| **Date** | 29 October 2026 |
| **Team ID** | Individual Project - Ujjawal Rathod |
| **Project Name** | Electric Vehicle Market Performance & Insights |
| **Maximum Marks** | 5 Marks |

# Data preparation workflow

The goal is to build a reliable progression from RAW source rows to STAGING and CURATED datasets for the EV market dashboard. Use a non-destructive approach: profile quality first, preserve raw data, document the business key for duplicate handling, and validate the counts after each transformation.

*SQL examples use a canonical curated schema. Map the source CSV headers to these aliases in the staging view before running: MODEL_YEAR, MARKET_REGION, VEHICLE_CATEGORY, MANUFACTURER, BATTERY_TYPE, DRIVING_RANGE_KM, ENERGY_CONSUMPTION_WH_KM, CUSTOMER_RATING, MANUFACTURING_COUNTRY, SALES_VOLUME. The supplied handoff did not include the raw CSV files or a live Snowflake connection, so numeric SQL outputs are intentionally not invented.*

## Activity 1: Identifying and Handling NULL Values

Profile nulls in required analytics fields. Do not automatically delete all records with any null; decide field-by-field. Exclude or quarantine records only when a missing value makes the requested KPI unusable.

### SQL

```
SELECT COUNT(*) AS TOTAL_ROWS,
    COUNT_IF(MODEL_YEAR IS NULL) AS NULL_MODEL_YEAR,
    COUNT_IF(VEHICLE_CATEGORY IS NULL) AS NULL_VEHICLE_CATEGORY,
    COUNT_IF(MANUFACTURER IS NULL) AS NULL_MANUFACTURER,
    COUNT_IF(BATTERY_TYPE IS NULL) AS NULL_BATTERY_TYPE,
    COUNT_IF(SALES_VOLUME IS NULL) AS NULL_SALES_VOLUME,
    COUNT_IF(DRIVING_RANGE_KM IS NULL) AS NULL_RANGE
FROM EV_RAW;
```

### Output and validation

Output: one row containing total rows and a null count per field. Record the actual returned counts; the project handoff describes 300,000 combined source rows before cleaning, which should be confirmed in Snowflake.

# Activity 2: Duplicate Record Handling

Inspect repeated rows before removal. Confirm the business key with the source documentation; if no stable vehicle-record identifier exists, review a composite key and retain raw records for traceability.

## SQL

```
SELECT HASH(*) AS ROW_HASH, COUNT(*) AS DUPLICATE_COUNT
FROM EV_RAW
GROUP BY ROW_HASH
HAVING COUNT(*) > 1
ORDER BY DUPLICATE_COUNT DESC;

-- Example after the business key is approved: deduplicate exact rows
CREATE OR REPLACE TABLE EV_STG_DEDUPED AS
SELECT DISTINCT *
FROM EV_RAW;
```

## Output and validation

Output: hashes and duplicate counts for repeated complete rows; the DISTINCT staging result retains one copy of identical rows. Compare row counts before and after and document why any business-level duplicates were removed.

# Activity 3: Data Cleaning and Standardization

Normalize text categories and manufacturer labels, convert numeric fields with TRY_TO_NUMBER/TRY_TO_DOUBLE, convert blank strings to NULL, and flag values outside valid ranges. Preserve raw source values for audit.

## SQL

```
CREATE OR REPLACE VIEW EV_STG_CLEAN AS
SELECT
  TRY_TO_NUMBER(MODEL_YEAR) AS MODEL_YEAR,
  NULLIF(UPPER(TRIM(MARKET_REGION)), '') AS MARKET_REGION,
  NULLIF(INITCAP(TRIM(VEHICLE_CATEGORY)), '') AS VEHICLE_CATEGORY,
  NULLIF(UPPER(TRIM(MANUFACTURER)), '') AS MANUFACTURER,
  NULLIF(UPPER(TRIM(BATTERY_TYPE)), '') AS BATTERY_TYPE,
  TRY_TO_DOUBLE(DRIVING_RANGE_KM) AS DRIVING_RANGE_KM,
  TRY_TO_DOUBLE(ENERGY_CONSUMPTION_WH_KM) AS ENERGY_CONSUMPTION_WH_KM,
  TRY_TO_DOUBLE(CUSTOMER_RATING) AS CUSTOMER_RATING,
  NULLIF(INITCAP(TRIM(MANUFACTURING_COUNTRY)), '') AS MANUFACTURING_COUNTRY,
  TRY_TO_DOUBLE(SALES_VOLUME) AS SALES_VOLUME
FROM EV_STG_DEDUPED;
```

## Output and validation

Output: a staging view with standardized category labels and safely converted numeric fields. Validate new null counts because failed TRY_ conversions become NULL rather than throwing errors.

# Activity 4: Data Transformation and Validation

Create a curated layer with derived fields and quality flags. Keep row-level exclusion criteria explicit; avoid silently discarding outliers without a business reason.

## SQL

```
CREATE OR REPLACE VIEW EV_CURATED AS
SELECT *,
      CASE WHEN CUSTOMER_RATING BETWEEN 1 AND 5 THEN TRUE ELSE FALSE END AS RATING_VALID,
      CASE WHEN MODEL_YEAR BETWEEN 2018 AND 2026 THEN TRUE ELSE FALSE END AS MODEL_YEAR_IN_SCOPE,
      CASE WHEN DRIVING_RANGE_KM > 0 THEN TRUE ELSE FALSE END AS RANGE_VALID
FROM EV_STG_CLEAN;

-- Validation summary
SELECT COUNT(*) AS CURATED_ROWS,
      COUNT_IF(MODEL_YEAR_IN_SCOPE) AS YEARS_IN_SCOPE,
      COUNT_IF(RATING_VALID) AS VALID_RATINGS,
      COUNT_IF(RANGE_VALID) AS VALID_RANGES
FROM EV_CURATED;
```

## Output and validation

Output: quality counts for curated rows, in-scope model years, valid ratings, and valid driving ranges. Store actual output screenshots after execution and compare source, staging, deduplicated, and curated counts.

# Validation gate before visualization

- Confirm total row count at every layer and explain any difference.
- Check that model years are in the intended analysis window (2018-2026) where required.
- Confirm customer ratings are within 1-5 and ranges/energy-consumption values are positive where applicable.
- Reconcile sales volume totals between CURATED and MART output views.